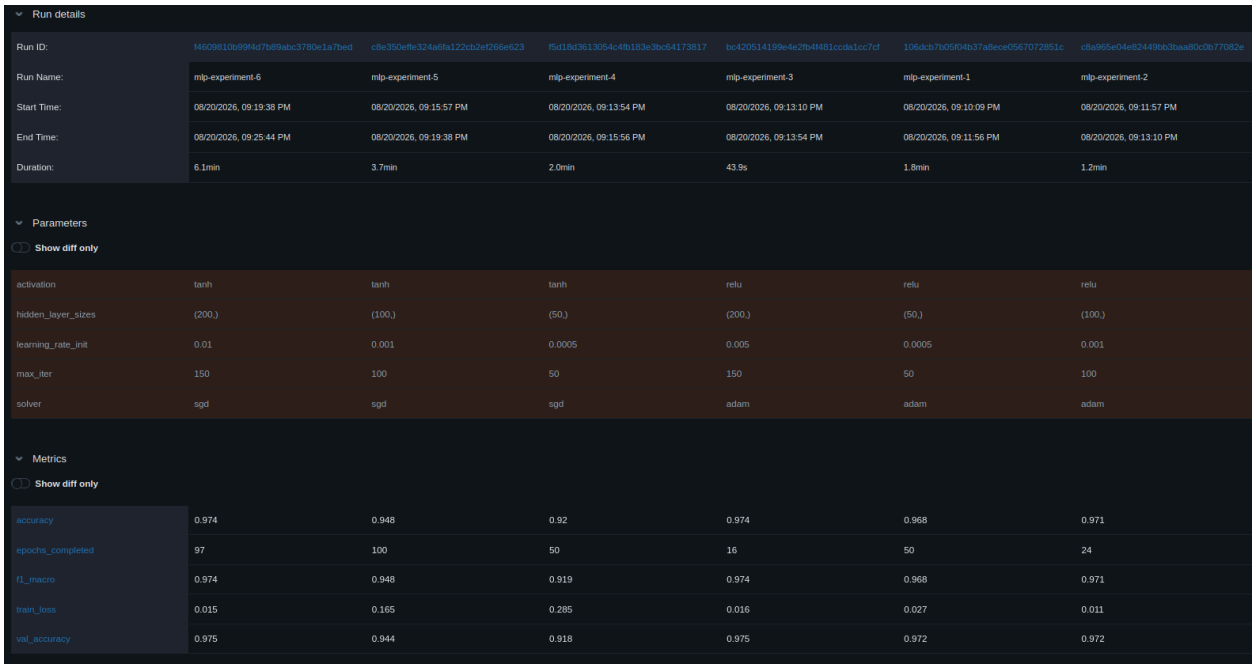
Question 2:

1.

	run_name	hidden_layer_sizes	activation	solver	learning_rate	max_iter	accuracy	f1_macro	run_id
0	mlp-experiment-3	(200,)	relu	adam	0.005	150	0.974357	0.974161	bc420514199e4e2fb4f481ccda1cc7cf
1	mlp-experiment-6	(200,)	tanh	sgd	0.01	150	0.974143	0.973894	f4609810b99f4d7b89abc3780e1a7bed
2	mlp-experiment-2	(100,)	relu	adam	0.001	100	0.971286	0.971086	c8a965e04e82449bb3baa80c0b77082e
3	mlp-experiment-1	(50,)	relu	adam	0.0005	50	0.968286	0.968060	106dcb7b05f04b37a8ece0567072851c
4	mlp-experiment-5	(100,)	tanh	sgd	0.001	100	0.948071	0.947699	c8e350effe324a6fa122cb2ef266e623
5	mlp-experiment-4	(50,)	tanh	sgd	0.0005	50	0.920214	0.919193	f5d18d3613054c4fb183e3bc64173817

```
Best run: mlp-experiment-3 (bc420514199e4e2fb4f481ccda1cc7cf) | accuracy=0.9744
```



Run details

Run ID:	f4609810b99f4d7b89abc3780e1a7bed	c8e350effe324a6fa122cb2ef266e623	f5d18d3613054c4fb183e3bc64173817	bc420514199e4e2fb4f481ccda1cc7cf	106dcb7b05f04b37a8ece0567072851c	c8a965e04e82449bb3baa80c0b77082e
Run Name:	mlp-experiment-6	mlp-experiment-5	mlp-experiment-4	mlp-experiment-3	mlp-experiment-1	mlp-experiment-2
Start Time:	08/20/2026, 09:19:38 PM	08/20/2026, 09:15:57 PM	08/20/2026, 09:13:54 PM	08/20/2026, 09:13:10 PM	08/20/2026, 09:10:09 PM	08/20/2026, 09:11:57 PM
End Time:	08/20/2026, 09:25:44 PM	08/20/2026, 09:19:38 PM	08/20/2026, 09:15:56 PM	08/20/2026, 09:13:54 PM	08/20/2026, 09:11:56 PM	08/20/2026, 09:13:10 PM
Duration:	6.1min	3.7min	2.0min	43.9s	1.8min	1.2min

Parameters

Show diff only

activation	tanh	tanh	tanh	relu	relu	relu
hidden_layer_sizes	(200,)	(100,)	(50,)	(200,)	(50,)	(100,)
learning_rate_init	0.01	0.001	0.0005	0.005	0.0005	0.001
max_iter	150	100	50	150	50	100
solver	sgd	sgd	sgd	adam	adam	adam

Metrics

Show diff only

accuracy	0.974	0.948	0.92	0.974	0.968	0.971
epochs_completed	97	100	50	16	50	24
f1_macro	0.974	0.948	0.919	0.974	0.968	0.971
train_loss	0.015	0.165	0.285	0.016	0.027	0.011
val_accuracy	0.975	0.944	0.918	0.975	0.972	0.972

2. The best performing run was experiment 3 (hidden_layer = 200, activation = relu, solver = adam, learning rate = 0.005, max_iter = 150) with both accuracy and f1 score of 0.974. This is because it was trained with the adam optimizer and ran for the maximum number of iterations compared to the other configurations. It also had the widest hidden layer dimension, allowing the MLP to capture more variance than the other model architectures. The training curves provide little evidence of serious overfitting. In both runs, training loss decreases throughout training, while validation accuracy generally improves before reaching a plateau. Although validation accuracy fluctuates slightly in later iterations, it does not show a sustained downward trend while training loss continues to fall. Therefore, the models appear to generalize reasonably well, with later training producing diminishing returns rather than clear overfitting.The learning rate appears to have one of the largest effects on performance, because increasing it from 0.0005 to higher values coincides with a substantial improvement in accuracy. Higher learning rates, combined with more iterations give higher accuracies.

3.

```
):
    with mlflow.start_run(run_name=run_name):
        mlflow.log_param("hidden_layer_sizes", str(hidden_layer_sizes))
        mlflow.log_param("activation", activation)
        mlflow.log_param("solver", solver)
        mlflow.log_param("learning_rate_init", learning_rate_init)
        mlflow.log_param("max_iter", max_iter)

        model = MLPClassifier(
            hidden_layer_sizes=hidden_layer_sizes,
            activation=activation,
            solver=solver,
            learning_rate_init=learning_rate_init,
            random_state=42,
            early_stopping=True,
            max_iter=max_iter,
        )

        model.fit(X_train, y_train)

        for step, (train_loss, val_accuracy) in enumerate(
            zip(model.loss_curve_, model.validation_scores_),
            start=1,
        ):
            mlflow.log_metric("train_loss", float(train_loss), step=step)
            mlflow.log_metric("val_accuracy", float(val_accuracy), step=step)

        mlflow.log_metric("epochs_completed", model.n_iter_)

        preds = model.predict(X_test)
        acc = accuracy_score(y_test, preds)
        f1 = f1_score(y_test, preds, average="macro")

        mlflow.log_metric("accuracy", acc)
        mlflow.log_metric("f1_macro", f1)
        mlflow.set_tag("team", "data-science")
        mlflow.sklearn.log_model(
            model,
            name="model",
            serialization_format="cloudpickle",
        )
```